

Common Type System (CTS) in .NET Framework

In this article, I am going to discuss the **Common Type System in .NET Framework**. Please read our previous article, where we discussed the [Intermediate Language \(ILDASM and ILASM\) in .NET Framework](#) with Examples. The Common Type System (CTS) in .NET Framework defines how data types are declared, used, and managed in the Common Language Runtime (CLR). At the end of this article, you will understand what is Common Type System (CTS) in C# and how CTS in .NET works.

What is the Common Type System in .NET Framework?

The .NET Framework supports many programming languages such as C#, VB.NET, J#, F#, etc. Every programming language has its own data type. One programming language data type cannot be understood by other programming languages. But, there can be situations where we need to communicate between two different programming languages. For example, we need to write code in the VB.NET language and that code may be called from C# language. In order to ensure smooth communication between these languages, the most important thing is that they should have a Common Type System (CTS) which ensures that data types defined in two different languages get compiled to a common data type.

CLR in .NET Framework will execute all programming language's data types. This is possible because CLR has its own data types which are common to all programming languages. At the time of compilation, all language-specific data types are converted into CLR's data type. This data type system of CLR is common to all .NET Supported Programming languages and this is known as the Common Type System (CTS) in .NET Framework.

Example to Understand Common Type System in .NET Framework

Let us understand Common Type System (CTS) in .NET Framework with an example. What we are going to do is, we will create two applications. One Application uses the C# Language and the other one is using VB.NET Language. And then we will try to see the IL code of both of these applications and then we will try to see what the CTS looks like.

Here we are going to create two class library projects. One class library project uses the C#.NET language and the other class library project uses the VB.NET language.

Creating C# Class Library Project:

Create a class library project with the name **CsharpClassLibrary** and use the **C#** programming language. Once you create the C# class library project then add a class file with the name **Calculator.cs** and then copy and paste the following code into it.

```
namespace CsharpClassLibrary
{
    public class Calculator
    {
        public int Calculate()
        {
            int a = 10, b = 20;
            int c = a + b;
            return c;
        }
    }
}
```

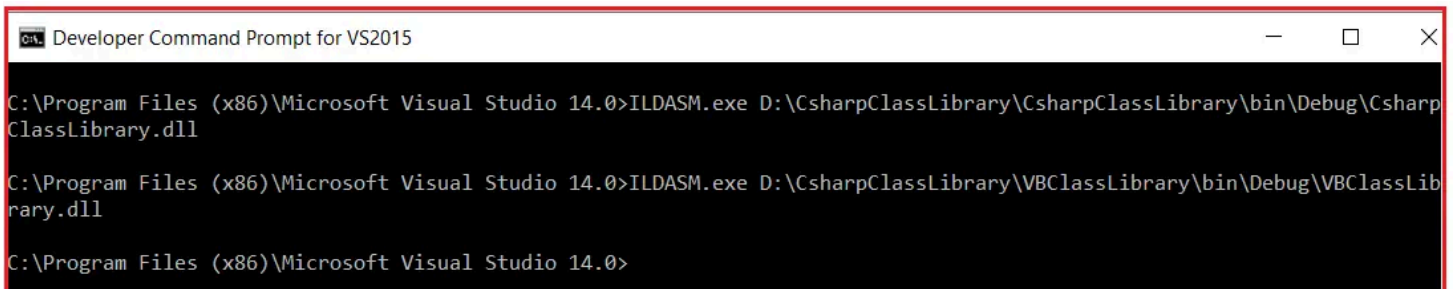
```
}  
}  
}
```

Creating VB Class Library Project:

To the same CsharpClassLibrary solution, let us add another class library project with the name as **VBClassLibrary** but using **VB** as the programming language. Once you created the VB Class library project then add a class file with the name **Calculator.cs** to this project and copy and paste the following code into it.

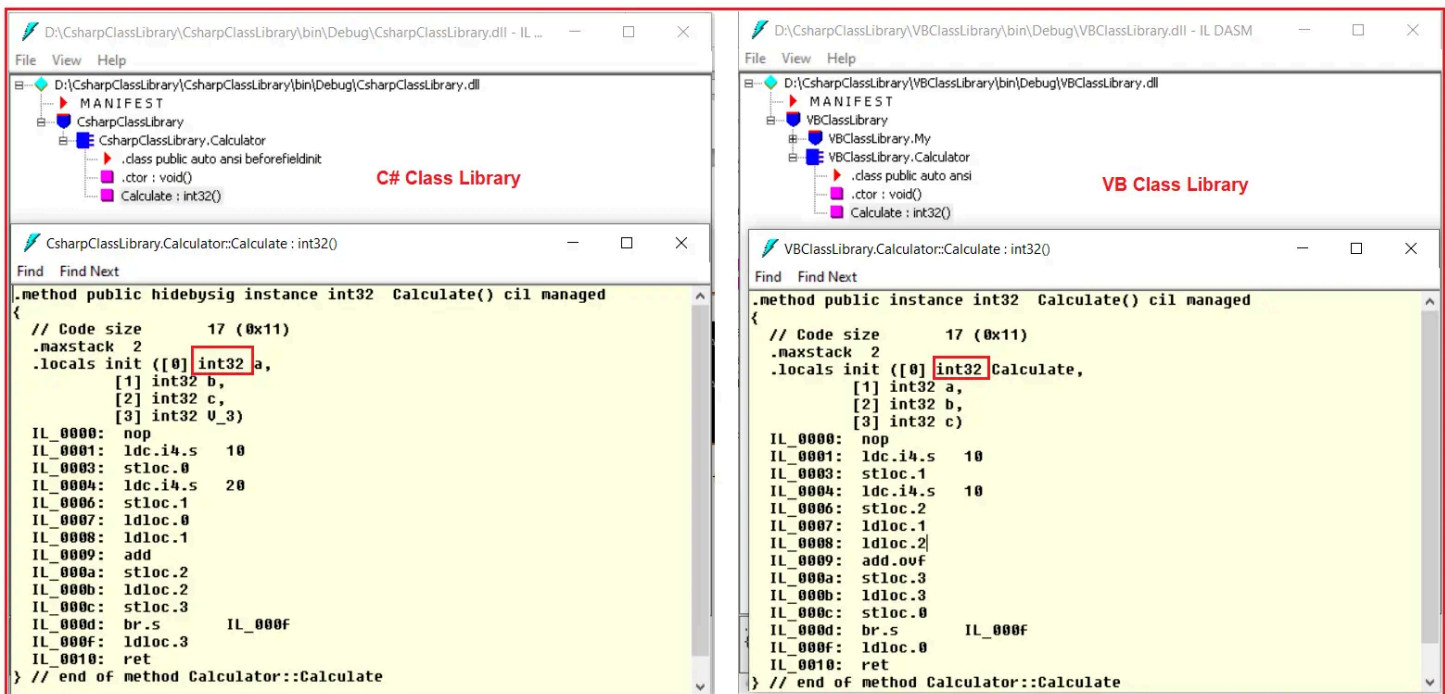
```
Public Class Calculator  
    Public Function Calculate() As Integer  
        Dim a As Integer = 10  
        Dim b As Integer = 10  
        Dim c As Integer  
        c = a + b  
        Return c  
    End Function  
End Class
```

Now, build the solution which should generate the respected .dlls. In our previous article, we discussed [how to use the ILDASM tool to see the IL code](#) in detail. So, let us open Visual Studio Command Prompt in Administrative mode and run two instances of the **ILDASM.exe** command i.e. one for the VB class library project DLL file and the other one for the C# class library project file as shown in the below image.



```
C:\Program Files (x86)\Microsoft Visual Studio 14.0>ILDASM.exe D:\CsharpClassLibrary\CsharpClassLibrary\bin\Debug\CsharpClassLibrary.dll  
  
C:\Program Files (x86)\Microsoft Visual Studio 14.0>ILDASM.exe D:\CsharpClassLibrary\VBClassLibrary\bin\Debug\VBClassLibrary.dll  
  
C:\Program Files (x86)\Microsoft Visual Studio 14.0>
```

Now, let us open the IL code of the Calculate method of both the class library project as shown in the below image. Please have a look at the integer variable in the IL code which is int32 in this case. In the C# class library project, we use int as the data type to declare variables a, b, and c whereas in the VB class library project, we use Integer as the data type to declare the variables a, b, and c. At the end of the day, both the data types are compiled to a common data type i.e. int32.



So, whether we write the code in VB.NET or in C#.NET, if it follows the dot net rules or specifications, at the end of the day it is compiled into a Common Type System (CTS) in .NET Framework which is common for all .NET Supported Programming Languages. This Common Type System is Provided by CLR and is the same for all .NET Supported programming languages. While compiling the source code by the respective language compiler, they will convert the language-specific data type to CLR-specific data type. For example, int in C# and Integer in VB is converted to Int32 after compilation and Int32 is the data type which is understood by CLR. Or you can say Int32 is the CLR-specific data type.

We already demonstrate IL code and CTS in C#. So, in the next article, I am going to demonstrate to you the example of **Common Language Specification (CLS)** in detail. Here, in this article, I try to explain the **Common Type System (CTS)** in .NET Framework in detail and I hope you enjoy CTS (Common Type System) in the C#.NET Framework article.